

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_47057438-9c9\agent-aa13cea5f0da98a00.jsonl`  
- SHA-256: `1cf9379d4d8e29d68a25b6b359428913669416467735bbc5f0ae436c71428f94`  
- Source modified: `2026-06-16T03:08:40+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_47057438-9c9`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T03:05:55.398Z)

DESIGN COMPONENT: The encryption / 'secret file' layer and KEY MANAGEMENT ? honest about what it buys. Compare git-crypt vs SOPS+age vs git-lfs(+encryption) vs keeping sensitive fixtures OUT of the repo entirely and fetching from a private store with a token. Design a TIERED model: Tier-0 = public, anonymized, UNENCRYPTED fixtures sufficient for pure-logic/characterization regression that ANY cloner can run with no key; Tier-1 = encrypted-in-repo or out-of-repo fixtures (richer/more-sensitive) for trusted contributors + CI who hold the key. Specify key distribution (CI secret, contributor onboarding), rotation, and what happens to a Tier-1 test when the key is absent (skip cleanly, not fail). Explicitly state that protobuf is serialization NOT secrecy, and that Tier-1 cannot hide content from a key-holder who runs it.

LEGAL MEMO (JSON):

```
{  
  "executiveSummary": "INTERNAL MEMO ? general legal information, NOT legal advice; retained India + EU/UK counsel sign-off required before any public commit. QUESTION: can the project commit DERIVED numeric data (per-frame shuttle (x,y) coordinates, rally start/end time-intervals, audio/person/court feature columns, ground-truth intervals) into a public GitHub repo so external contributors can run the rally-segmentation regression suite WITHOUT the videos? BOTTOM LINE: Yes, but only under conditions, and the analysis FORKS on data provenance. The OUTPUT data itself is, across all three jurisdictions, essentially uncopyrightable FACT/measurement ? it does not inherit the source video's copyright (US: Feist + 17 USC 102(b) + NBA v. Motorola; India: EBC v. D.B. Modak, no sui generis right; EU: Football Dataco v. Yahoo ? copyright protects only original selection/arrangement, never the numbers). So a thin compilation copyright at most attaches to a creative schema, never to the coordinates. BUT copyright is the easy axis and NOT where the risk lives. The real exposure is fourfold and provenance- and stream-dependent: (1) the ACT of ingesting/extracting from the source video needs its own lawful basis ? owned/permissioned video is clean; a US fair-use defense for analytic extraction is now CONTESTED, not \"strongly favorable,\" after Thomson Reuters v. Ross (2025) and the US Copyright Office May-2025 report, and unlawful acquisition independently destroys the defense (Bartz v. Anthropic, ~$1.5B settlement); (2) CONTRACT / Terms-of-Service can bar extraction and redistribution even where copyright cannot reach the facts (ProCD; hiQ v. LinkedIn; YouTube ToS bars scraping AND \"harvest information that might identify a person (for example... faces)\") AND redistribution) ? and the remedy is injunction + forced deletion of the committed dataset, not nominal damages; (3) EU/UK sui generis DATABASE RIGHT may subsist in self-derived match-event/coordinate streams because, on the leading Football Dataco v. Sportradar reasoning, recording pre-existing on-court facts you do not control is \"OBTAINING,\" not \"creating\" ? so the project's comfortable \"we created it\" premise is genuinely contestable; (4) PRIVACY ? person/pose/gait streams and anything involving MINORS are the highest-risk and several should NOT be published at all. The cleanest defensible path: commit ONLY owned-or-permissioned-source, minors-excluded, person/pose/gait-EXCLUDED (shuttle + abstract rally-interval + non-biometric feature) data, stripped of every source-attribution field (no filename, no video_id MD5, no match metadata), under a no-extraction-from-third-party-source and no-redistribution hygiene posture, with optional encryption only as a barrier against the no-key public (it cannot bind a contributor who runs the tests).\",  
  "derivedDataCoreVerdict": "The derived numeric data DOES NOT inherit the source video's
```

copyright in any of the three jurisdictions ? this is the one firmly-settled conclusion. Per-frame shuttle (x,y) coordinates, rally start/end timestamps, and audio/person/court feature values are uncopyrightable FACTS/measurements about a recorded physical event: discovered, not authored. US: Feist (facts never copyrightable; \"originality, not sweat of the brow, is the touchstone\"), 17 USC 102(b) (no copyright in any \"discovery\"), NBA v. Motorola (facts generated by a sporting event are uncopyrightable \"purely factual information which any patron... could acquire from the arena\" even though the broadcast is protected). A coordinate/timestamp table is also NOT a 17 USC 101 \"derivative work\": a derivative work recasts the original's protected EXPRESSION (framing, editing, imagery, commentary), and a number table carries none of it. India: EBC v. D.B. Modak ? copyright needs \"skill and judgment,\" protects only original arrangement, not facts; no EU-style sui generis right exists. EU: Football Dataco v. Yahoo (C-604/10) ? database copyright protects only an original selection/arrangement; \"the intellectual effort and skill of creating that data are not relevant\" and sweat-of-the-brow cannot ground it. CEILING ON PROTECTION: the MOST that can attach is a THIN compilation copyright in a genuinely creative SELECTION/ARRANGEMENT (e.g., a curated schema) ? never in the raw numbers, and a mechanical/exhaustive layout (every frame, every rally) likely has too little originality even for that (Feist's \"garden-variety listing\" problem). CRITICAL SEPARATION: the STATUS of the output data (settled: unprotected facts) is a different question from the ACT of extraction (transiently copying the video to compute the data), which can independently infringe (Sega v. Accolade: intermediate copying can infringe even when the end product reproduces no protected expression) and which needs its own license or fair-use basis. NON-COPYRIGHT FENCES survive precisely BECAUSE the data is uncopyrightable: enforceable no-extraction ToS (ProCD), narrow hot-news misappropriation (NBA v. Motorola), and the EU/UK sui generis database right. So \"the numbers are free facts\" answers the copyright question and ONLY the copyright question.

```
"byJurisdiction": [
{
  "jurisdiction": "India (PRIMARY ? owner is Bangalore-based, commercialization intended)",
  "copyright": "Derived facts are NOT copyrightable as such: EBC v. D.B. Modak (2008 SC) requires 'skill and judgment' and protects only original arrangement, not facts; India has NO EU-style sui generis database right. BUT this is not as clean as it looks: Indian High Courts protect DATABASES/COMPILATIONS as 'original literary works' under s.2(o) on a low 'skill, labour and judgment' threshold ? Burlington Home Shopping v. Rajnish Chibber, V. Govindan v. E.M. Gopalakrishna Kone, and the OLX v. Padawan order itself (which held a scraped dataset to be a protectable 'proprietary database'). So if numbers are LIFTED from a third party's structured dataset/feed, copying its selection/arrangement can infringe even though the raw facts are free. Self-derived-from-pixels data avoids this; copying an official stats compilation does not.",
  "databaseRight": "No sui generis database right exists in India ? a genuine permissiveness advantage over the EU/UK. Protection runs only through copyright (compilation, above) or contract/IT-Act.",
  "contractToS": "The LIVE risk, not copyright. Breach of anti-scraping ToS plus IT Act s.43 (Government has affirmatively argued public-data scraping 'may be in breach of section 43') and s.66 layering CRIMINAL exposure (up to 3 years / Rs 5 lakh). Enforceability of anti-scraping ToS is untested in Indian courts. OLX v. Padawan was decided AGAINST the scraper; its 'private use' line is obiter, not a safe harbor. Trade-secret/confidentiality for the owner's OWN corpus runs NOT through any US-style DTSA (India has no trade-secret statute) but through the Indian Contract Act 1872, equity, common-law breach of confidence, and IT Act s.72 ? and publication still forfeits the moat under that doctrine.",
  "privacy": "DPDP Act 2023 governs personal data of identifiable natural persons; genuinely anonymized numbers fall outside it, but the s.3(c)(ii) 'publicly available data' carve-out is CONTESTED (MeitY/Minister has said scrapers must still meet DPDP consent obligations). MINORS are stricter than the EU: 'child' = under 18 (s.2(f)); s.9(1) requires VERIFIABLE parental consent (DPDP Rules 2025 Rule 10 ? real ID/age verification, not a checkbox); s.9(2) bars detrimental processing; s.9(3) PROHIBITS tracking/behavioural monitoring/targeted ads of children ? subject to narrow Rule 12/Fourth-Schedule class/purpose exemptions that do NOT cover an amateur highlight indexer, so the ban DOES bite this project. A persistent per-player pose/gait signature is, on the conservative reading, prohibited behavioural monitoring; 'tracking' is undefined in the Act, so this is the defensible-conservative, not the certain, reading.",
  "netVerdict": "MORE PERMISSIVE than the EU on the database-right axis, but NOT 'lawful on its face.' Defensible to publish ONLY genuinely-anonymized, independently-derived (not lifted from a third-party compilation) numbers, with NO minors tracked and accepting that anti-scraping-ToS + IT Act s.43/s.66 exposure for any third-party-sourced ingestion is unresolved. Owned-source + minors-excluded + no-person/pose/gait = clean."
},
```

```

{
  "jurisdiction": "United States (repo + product are global; US fair-use & contract law are
the most-developed)",
  "copyright": "Output data does not inherit the video's copyright (Feist; 102(b); NBA v.
Motorola); not a 101 derivative work; thin compilation copyright at most. The EXTRACTION act is
the open question: fair use for analytic, non-expressive copying was historically favorable
(Authors Guild v. Google; Google v. Oracle) but is now CONTESTED ? Thomson Reuters v. Ross (D.
Del. 2025) REJECTED the 'copy an expressive work just to reach unprotected facts/data' theory,
distinguished the functional-CODE cases (Sega/Oracle), found the use non-transformative, and
found Factor-4 harm in a derivative 'data to train AIs' market the plaintiff had not even
entered; the US Copyright Office (May 2025) rejected 'non-expressive therefore inherently
transformative' and backs a market-dilution harm theory. If the dataset competes with a rights-
holder's own data/analytics market, fair use may FAIL. Unlawful source acquisition independently
defeats the defense regardless of how transformative the end use is (Bartz v. Anthropic, 2025;
~$1.5B settlement).",
  "databaseRight": "No sui generis database right in the US (Feist rejected sweat-of-the-
brow). Competitors stay free to copy raw facts ? subject to the non-copyright fences below.",
  "contractToS": "The dominant US risk. Post-Van Buren the CFAA no longer reaches mere
purpose-violations of accessible/public data (though footnote 8 RESERVED whether contract/policy
limits can still ground CFAA liability ? the federal-crime route is narrowed, not dead), so
platforms enforce via CONTRACT. ToS is an enforceable contract that bites where copyright/CFAA
fail (ProCD; hiQ v. LinkedIn ? breach-of-contract summary judgment + $500k consent judgment +
injunction + ORDER TO DESTROY scraped-derived data). Binding turns on assent: a logged-
in/account/API-key or genuine clickwrap user is bound (hiQ; any YouTube-API user); a purely
logged-out scraper under browwrap-only may not be (Meta v. Bright Data) ? and thin 'sign-in
wrap' flows can fail for want of conspicuous notice. YouTube ToS verbatim bars automated access,
'harvest any information that might identify a person (for example...faces)', and
redistribution/commercial reuse. REDISTRIBUTION (committing the dataset publicly) is the single
highest-risk act and the remedy is injunction + forced deletion, not nominal damages.",
  "privacy": "GDPR-style analysis below applies to global users; US state biometric statutes
(e.g., Illinois BIPA ? not separately briefed here) and publicity/personality rights are
additional fences for face crops and identifiable-player gait. Faces in any committed imagery =
identifying personal data; do not publish.",
  "netVerdict": "Output data publishable as facts ONLY if (i) source video was
owned/permissioned (or extraction has a defensible fair-use basis you do NOT over-rely on), (ii)
no enforceable ToS bars extraction/redistribution of the ingested source, and (iii) no
person/pose/gait/face/minor stream is included. Treat third-party-ToS-sourced derived data as
NOT publicly committable."
},
{
  "jurisdiction": "EU / UK (product/repo global; English-language highlight market makes UK
a likely target)",
  "copyright": "Database copyright protects only an original selection/arrangement, never
the numbers, and never sweat-of-the-brow (Football Dataco v. Yahoo, C-604/10). Self-derived
coordinates are facts, not a copied original structure ? so copyright is a low axis.",
  "databaseRight": "The genuinely contestable axis. The sui generis right protects
investment in OBTAINING/verifying/presenting pre-existing data, NOT in CREATING it (BHB v.
William Hill C-203/02; Fixtures Marketing) ? which the project reads as 'we create coordinates
frame-by-frame, so we are clear.' THAT READING IS OVERCONFIDENT: in Football Dataco v.
Sportradar the live match-event data (goals/scorers) was held to be OBTAINED, because the
recorder captured pre-existing facts ('events taking place on the pitch') it did NOT control. A
badminton rally/shuttle position is the same kind of pre-existing on-court fact recorded rather
than authored, so a rights-holder can argue the right SUBSISTS in our streams. Infringement is
now governed by CV-Online Latvia (C-762/19, 2021): even a substantial-part taking is actionable
only where it adversely affects the maker's ability to recoup investment, balanced against
information flow/competition ? a fact-intensive harm test, not a clean 'no protected part =
safe' rule. Sportradar (C-173/11) also defeats the server-location dodge for EU-targeted re-
utilisation. UK retained its own sui generis right and pre-2021 CJEU case law, so UK-targeted
products carry AT LEAST equal, arguably greater, risk; a non-UK/EEA maker may not even hold the
right defensively, which does nothing to reduce exposure as a potential infringer.",
  "contractToS": "Ryanair v. PR Aviation (C-30/14): where a database enjoys NEITHER
copyright NOR the sui generis right, the Directive's user-protections (incl. no-contracting-out)
do not apply, so the source's T&Cs/licence can lawfully BAN automated extraction and
redistribution and bind anyone who agreed ? the dominant residual risk. Trap: the licence that
legitimises INGESTION of the footage is often the very instrument that contractually bans

```

extraction/redistribution of derived data.",

"privacy": "GDPR: shuttle/object coordinates are NOT personal data in isolation (Art.4(1)) ? but coordinates published as part of an identifiable named match are part of a personal-data operation. Person bounding boxes that single out a human are ordinary personal data, NOT Art.9 biometric UNLESS used to uniquely identify (EDPB Guidelines 3/2019). Pose/gait is the danger zone: gait re-identifies even with faces blurred and skeleton 'anonymization' leaves gait identity intact (Weiss et al., Sensors 2025) ? Art.9 special-category the moment used to uniquely identify; explicit-consent basis required. 'Anonymous' is RELATIVE post EDPS v. SRB (CJEU C-413/23 P, 4 Sept 2025): a stream can be non-attributable to a recipient lacking the source yet remains PERSONAL DATA to the holder who can re-run the detector. Minors: Art.8 (consent floor 13?16); any biometric/special-category processing of a child stacks Art.9.",

"netVerdict": "Do NOT assume self-derivation defeats the database right ? treat self-derived event/coordinate streams as potentially 'obtained' data and avoid taking in a way that could reconstitute a rights-holder's dataset or harm their feed's monetisation. Read every source's T&Cs for extraction/redistribution clauses. Keep the footage-copyright analysis (highlight reels) strictly separate. Person/pose/gait/face = do not publish without an Art.9 explicit-consent basis. Get EU + UK jurisdiction-specific advice before redistributing numeric streams at scale."

}

1,

"provenanceFork": "PROVENANCE IS THE DECISIVE FORK ? the legal posture of an identical numeric file flips entirely on where the source video came from. FORK A ? OWNED / OWNER-RECORDED (or fully-permissioned/licensed-for-this-use) SOURCE: This is the clean path and the only one safe to commit publicly. No third-party copyright in the source bars the extraction; no third-party ToS governs the owner's own recording (with two caveats below); the EU/UK database-right 'obtained vs created' debate still applies in theory but there is no adverse rights-holder to assert it against the owner's own footage. The analysis collapses to: (1) a TRADE-SECRET / competitive-moat CHOICE (US framing: 18 USC 1839 DTSA requires secrecy measures + value-from-secrecy; the OPERATIVE framing for this Bangalore owner is Indian breach-of-confidence / Contract Act 1872 / IT Act s.72, NOT the DTSA) ? publishing the data forfeits whatever moat it carried, which is the owner's strategic call to make, weighed against the stated community/benchmark/paper-co-authorship aims; and (2) PRIVACY ? the owner's own footage still contains identifiable people, so DPDP/GDPR and the minors rules below still bind, and person/pose/gait/face streams are still off-limits. TWO CAVEATS that narrow Fork A: (a) 'owner-recorded' is clean ONLY for the owner's OWN non-ticketed/private sessions; footage the owner SHOT AT a third party's event (tournament, club match) carries live exposure ? a ticket is a contract of admission that routinely bars recording/redistribution, and venue/organiser/broadcaster rights (e.g., BWF/league terms, Rome Convention rebroadcast rights) attach independently of any platform ToS; (b) any third-party-collected footage in the corpus (the project's golden set may mix sources) taints those records and pushes them into Fork B. FORK B ? THIRD-PARTY-SOURCED (broadcast / YouTube / scraped / downloaded) SOURCE: Derived data from this provenance SHOULD NOT be committed to a public repo. Even though the numbers are uncopyrightable facts, (1) the EXTRACTION act needs a license or a now-CONTESTED US fair-use defense, and unlawful acquisition (e.g., scraping against ToS, downloading from a platform that bars it) independently sinks the defense (Bartz v. Anthropic); (2) the source's ToS (YouTube et al.) typically bars automated extraction AND redistribution, and committing the derived dataset publicly IS redistribution ? the highest-risk act, remediable by injunction + forced deletion of the committed data (hiQ); (3) re-utilisation/extraction of a substantial part may infringe the EU/UK sui generis database right if the source ties back to a protected feed. PRACTICAL RULE FOR THE REPO: gate the committed corpus to FORK A only. Tag every golden-set record with a verified provenance flag at ingestion; only owned/permissioned-source, non-event-ticketed, minors-excluded, person/pose/gait-excluded records are eligible for the public commit. This maps directly onto the existing split: eval_baselines/ (aggregate JSON snapshots) is already committed precedent and is provenance-agnostic if it carries no per-source attribution; *_golden_features.json are currently kept private (docs/GOLDEN_DATA_SHARING.md) precisely because they derive from a 'Proprietary corpus' ? the right move is to keep Fork-B-tainted features private and publish only a Fork-A, de-attributed subset.",

"privacyVerdict": "PUBLISH/NO-PUBLISH BY STREAM (the project's guardrails already point the right way; this hardens them). PUBLISH-OK (lowest risk): (1) SHUTTLE (x,y) coordinates + Visibility ? describe an inanimate object, NOT personal data on their own (GDPR Art.4(1); WP136 content/purpose/result test) ? PROVIDED they are NOT joined to anything that singles out a named player and NOT published as an identifiable named match (the surrounding personal-data obligations survive even when the coordinate column alone is object-data). (2) Abstract RALLY start/end time-INTERVALS and ground-truth intervals stripped of match/player identity ? facts about event timing. (3) Non-biometric AUDIO-ONSET, COURT-BOUNDS, and aggregate PERSON

COUNT/feature columns used ONLY for rally detection and DETACHED from any identity ? ordinary low-risk data with normal hygiene; NOT Art.9 because the purpose is detect/categorize, not uniquely identify (EDPB Guidelines 3/2019). CONDITIONAL ? only with an Art.9 explicit-consent basis (and effectively never for this v1): PERSON bounding-box coordinates that persistently single out and follow a specific player across rallies ? the purpose flips to unique identification and Art.9 attaches; this architecture is PRONE to that flip, so treat per-player tracking as a tripwire. DO NOT PUBLISH (high risk, exclude from any committed stream): (4) POSE keypoints / SKELETON / GAIT signatures ? gait is a behavioural biometric that re-identifies people even with faces blurred, and pose 'anonymization' provably leaves gait identity intact (Weiss et al., Sensors 2025, 80?95% gait-recognition on anonymized data); raw pose/gait tracks are NOT reliably anonymous, are personal data when usable to single out a player, and become Art.9 special-category once used to uniquely identify ? this matches the project's existing 'gait/biometrics strictly FUTURE (GDPR Art.9)' guardrail. (5) Any FACE CROP or identifiable IMAGERY ? identifying personal data under GDPR and DPDP, and barred by YouTube ToS's face-harvesting clause; do not commit imagery at all. (6) Anything that joins a NAMED PLAYER to a gait/pose signature. MINORS FLIP EVERYTHING: under DPDP a 'child' is anyone under 18; processing ANY child's data needs VERIFIABLE parental consent (Rule 10 ? real ID/age check, not a checkbox), must not cause detrimental effect, and s.9(3) PROHIBITS behavioural monitoring/tracking of children (Rule 12/Fourth-Schedule exemptions do not cover this project, so the ban applies). A persistent per-player pose/gait signature is, conservatively, prohibited monitoring. CLEANEST PATH for any footage containing minors: EXCLUDE minors entirely from any persisted derived stream beyond inanimate shuttle/interval data ? consistent with the existing 'EXCLUDE MINORS from the training pool' guardrail ? or obtain Rule-10 verified parental consent and keep only de-identified object/coordinate data. NON-ATTRIBUTABILITY ARCHITECTURE (the 'secret file' question): Non-attributability is a property of the (data, holder, environment) triple, not of the timeseries. A per-video trajectory/feature stream is effectively a FINGERPRINT (de Montjoye: 4 points re-identify 95% of 1.5M people; k-anonymity collapses on rich quasi-identifiers), so a deterministic detector re-run on a candidate video can correlate-match it to its source ? BUT only if a correlatable reference EXISTS. PUBLIC source: a permanent cheap reference exists, re-linkage is 'reasonably likely', so a raw public-sourced stream should be treated as ATTRIBUTABLE; only heavy aggregation, perturbation/differential privacy, synthetic data, or a non-public enclave (NIST SP 800-188) breaks correlatability. PRIVATE/owner-recorded-and-unpublished source: there is no external reference to correlate against, so source-linkage by an EXTERNAL adversary is not 'reasonably likely' ? BUT it is WRONG to call this 'anonymous by default' (EDPS v. SRB, CJEU C-413/23 P, 4 Sept 2025): to the holder who can re-run the detector, it REMAINS personal data; it is merely RELATIVELY non-attributable to outsiders, and that protection is conditional, revocable (publish or leak the source and the link snaps back), and forward-looking (regulators say assume the data environment grows). HARD ENGINEERING IMPLICATION: the committed fixtures must NOT carry the source filename, match metadata, or the video_id (which is the MD5 of the source file ? using it as a public key would FINGERPRINT/confirm the source the instant anyone holds the file). Use an opaque, random, non-reversible surrogate id per record. Strip the 'name' field in *_golden_features.json. The lead's established constraint is correct: a contributor who RUNS the tests can necessarily read the plaintext fixtures and any key on their machine (DRM/analog-hole limit), so secrecy-from-the-contributor is impossible; the achievable goal is NON-ATTRIBUTABILITY + uselessness-in-isolation, with encryption useful ONLY as a barrier against the no-key public, never against a running contributor.",

"conditionsForSafePublicCommit": [

"PROVENANCE GATE: commit ONLY derived data from owner-recorded / fully-permissioned source video (Fork A). Tag every golden-set record with a verified provenance flag at ingestion; exclude any third-party-sourced (broadcast/YouTube/scraped/downloaded) record from the public commit. For owner-SHOT footage of a third party's event, first clear the admission ticket / venue / organiser / broadcaster terms ? only owner's own non-ticketed sessions are unconditionally clean.",

"SOURCE-LICENCE/ToS CHECK: for every source that touched the committed corpus, confirm in writing that its licence/ToS does NOT bar automated extraction OR redistribution of derived data. Do not rely on US fair use for the extraction step ? it is now contested (Thomson Reuters v. Ross; USCO May-2025) and unlawful acquisition independently defeats it (Bartz v. Anthropic). Redistribution (the public commit) is the single highest-risk act.",

"MINORS EXCLUSION: exclude all footage containing minors (under 18, per DPDP s.2(f)) from any persisted/committed derived stream beyond inanimate shuttle/interval data ? unless Rule-10 verifiable parental consent is held AND only de-identified object/coordinate data is kept. s.9(3)'s behavioural-monitoring ban means a persistent per-player pose/gait signature on a minor is prohibited.",

"BIOMETRIC EXCLUSION: do NOT publish pose keypoints, skeleton tracks, gait signatures, face

crops, identifiable imagery, or any named-player-to-movement-signature mapping. Restrict the committed feature columns to shuttle (x,y)+Visibility, abstract rally/GT time-intervals, and non-biometric audio/court/aggregate-person features used only for rally detection and detached from identity.",

"DE-ATTRIBUTION: strip every source-identifying field from the committed fixtures ? source filename, match/event/player metadata, capture dates, and especially the video_id (MD5 of the source file). Replace with an opaque, random, non-reversible surrogate record id. The 'name' field in *_golden_features.json must be removed/aliased.",

"NON-ATTRIBUTABILITY POSTURE: because per-video trajectories are fingerprints correlatable to a source video, achieve non-attributability by keeping the SOURCE private (Fork A, unpublished) ? which makes external source-linkage not 'reasonably likely' ? and/or by coarsening/aggregating/perturbing where feasible. Document the (data, holder, environment) assessment and re-test it as the public-video environment grows (ICO/OVIC forward-looking duty). Treat the streams as personal data IN THE OWNER'S OWN HANDS regardless (EDPS v. SRB).",

"USELESS-IN-ISOLATION + ENCRYPTION-AS-PUBLIC-BARRIER ONLY: keep the committed fixtures useless without the regression harness + schema (abstract intervals + opaque ids, no imagery, no source pointer). Use encryption only as a barrier against the no-key public; do not represent it as protecting against a contributor who runs the tests (they can read plaintext + key locally ? the lead's established DRM/analog-hole constraint).",

"PRECEDENT-CONSISTENT PLACEMENT: publish via the already-committed eval_baselines/ pattern (aggregate, provenance-agnostic JSON snapshots) for headline regression baselines; keep Fork-B-tainted or biometric-bearing *_golden_features.json private (per docs/GOLDEN_DATA_SHARING.md) and publish only a Fork-A, de-attributed, minors-excluded, biometric-free subset of features needed to run the suite.",

"DPDP HYGIENE: ensure any person-derived numeric column that could single out a player is genuinely identity-detached; do not rely on the DPDP s.3(c)(ii) 'publicly available data' carve-out for non-anonymized personal data (it is contested). Maintain purpose-limitation: features persisted for rally detection must not be repurposed for player identification.",

"COUNSEL SIGN-OFF: obtain retained India counsel (DPDP s.9/Rules, IT Act s.43/s.66, breach-of-confidence) AND EU/UK counsel (sui generis database right 'obtained vs created', CV-Online harm test, Ryanair contract) review of the final committed subset and the source-provenance log BEFORE the first public commit. This memo is general information, not advice."

],

"redFlags": [

"OVERCONFIDENCE TRAP #1 ? 'we created the data, so the EU/UK database right cannot apply.' Football Dataco v. Sportradar held live match-event data was OBTAINED (pre-existing on-court facts the recorder did not control), exactly analogous to a recorded badminton rally/shuttle position. A rights-holder can credibly argue the sui generis right subsists in our self-derived streams. Do not bank on the 'created' characterisation.",

"OVERCONFIDENCE TRAP #2 ? 'analytic extraction is strongly favorable fair use (US).' Thomson Reuters v. Ross (2025) REJECTED the 'copy an expressive work to reach unprotected facts' theory and found Factor-4 harm in a derivative data market; the USCO (May 2025) rejected 'non-expressive = inherently transformative.' If the dataset competes with a rights-holder's data/analytics market, fair use may fail. The extraction step is litigable, not settled.",

"OVERCONFIDENCE TRAP #3 ? 'owner's own recordings have no contract problem.' False for footage shot at a third party's event: the admission ticket is a contract barring recording/redistribution, and venue/organiser/broadcaster rights (BWF/league/Rome Convention) attach independently. Only the owner's own non-ticketed sessions are unconditionally clean.",

"OVERCONFIDENCE TRAP #4 ? 'private/unpublished source = anonymous by default.' Per EDPS v. SRB (CJEU 2025) the stream REMAINS personal data to the holder who can re-run the detector; it is only RELATIVELY non-attributable to an external adversary lacking the source, and that is conditional, revocable, and forward-looking. Misclassifying it as 'anonymous' is the exact GDPR error the controller's-perspective holding exists to prevent.",

"FINGERPRINTING / video_id leak: the project's video_id is the MD5 of the source file. Committing it (or any source filename/match metadata) as a public key would let anyone holding the candidate video CONFIRM the source instantly ? defeating non-attributability. Must be stripped and replaced with an opaque surrogate.",

"REDISTRIBUTION REMEDY is not nominal damages: the operative relief in scraping/ToS disputes is INJUNCTION + FORCED DELETION of the derived dataset (hiQ ordered destruction of all scraped-derived code/data). A public commit that breaches a source ToS can be ordered taken down and deleted ? a real, not theoretical, downside.",

"POSE/GAIT mislabeled low-risk: skeleton 'anonymization' leaves gait identity intact (80?95% re-ID), so any pose/gait stream is NOT reliably anonymous and is Art.9-adjacent. Excluding it is not optional polish ? it is a guardrail.",

"MINORS + behavioural monitoring: DPDP s.9(3) bars behavioural monitoring of under-18s and

the Rule 12/Fourth-Schedule exemptions do not cover this project. A persistent per-player pose/gait signature on a minor is, conservatively, a prohibited act with criminal-adjacent regulatory exposure ? exclude minors from any persisted stream beyond inanimate shuttle/interval data.",

"MIXED-PROVENANCE corpus: the golden set may blend owned and third-party footage. A single Fork-B-tainted record contaminates the public commit. Provenance must be verified per-record, not assumed corpus-wide.",

"WRONG-LAW trade-secret framing: analysing the owner's moat under US 18 USC 1839 (DTSA) is a jurisdictional error ? the Bangalore owner's protection runs through Indian breach-of-confidence / Contract Act 1872 / IT Act s.72. The publish-forfeits-the-moat intuition survives, but the operative doctrine is different."

],

"counselQuestions": [

"EU/UK sui generis database right: on the Football Dataco v. Sportradar 'obtained vs created' reasoning, does the right subsist in our self-derived shuttle-coordinate / rally-event streams, and if so does redistributing a de-attributed regression subset 'extract/re-utilise a substantial part' and cause CV-Online-Latvia-type harm to any rights-holder's feed? (EU AND UK, given likely English-language market.)",

"US fair use for the EXTRACTION step, post-Thomson Reuters v. Ross and the USCO May-2025 report: is transient analytic copying of a source video to compute rally/coordinate features defensible where the resulting dataset could be seen to compete in a 'data/analytics' market ? and does it matter that we only ever publish numbers, never the footage?",

"India: are anti-scraping/ToS terms enforceable in Indian courts, and what is the realistic IT Act s.43 (civil) / s.66 (criminal) exposure for ingesting third-party footage to derive features ? and does the pending ANI v. OpenAI judgment (reserved, ~April 2026) change the fair-dealing/TDM analysis?",

"India DPDP s.9 + Rules 10/12: for a corpus that may contain players under 18, what verifiable-parental-consent and exemption analysis applies, and does a persistent per-player pose/gait (or even per-player rally-tracking) feature constitute prohibited 'behavioural monitoring/tracking' of a child under s.9(3)?",

"Owner-shot event footage: do the admission tickets / venue / organiser (e.g., BWF, local league) / broadcaster terms for the specific tournaments and clubs in our corpus bar recording, derivation, or redistribution of derived data ? and which records must therefore be excluded from the public commit?",

"Personal-data classification under GDPR + DPDP for the SPECIFIC committed feature columns: at what point do identity-detached person/court/coordinate features 'single out' a player (especially in a small known squad/roster), and does our de-attribution + opaque-surrogate-id scheme suffice to keep the published subset outside personal-data obligations for downstream recipients (per EDPS v. SRB)?",

"Trade-secret / breach-of-confidence (Indian law): given commercialization and possible paper co-authorship, what is the moat trade-off of publishing the Fork-A subset, and are there confidentiality/Contract-Act 1872 / IT Act s.72 steps to take before forfeiting secrecy on any owner-recorded derived data?",

"Source-licence audit: for each source category in the corpus (owner-recorded, permissioned partner, downloaded/scraped), confirm the licence/ToS expressly permits BOTH automated extraction AND public redistribution of derived numeric data ? and where silent or prohibitive, which records are ineligible.",

"Encryption + 'secret file' design: does adding encryption that a running contributor can necessarily decrypt create any misrepresentation/false-security or DPDP 'reasonable security safeguards' compliance question, and is non-attributability-via-private-source a defensible posture to assert to regulators given its conditional/revocable nature?",

"Cross-border data transfer: does committing any record that is (or may be argued to be) personal data to a global public GitHub repo implicate DPDP cross-border-transfer rules or GDPR international-transfer obligations, independent of the copyright/contract analysis?"

],

"citations": [

{

"name": "Feist Publications, Inc. v. Rural Telephone Service Co., 499 U.S. 340 (1991) ? Justia",

"url": "https://supreme.justia.com/cases/federal/us/499/340/"

},

{

"name": "National Basketball Ass'n v. Motorola, Inc., 105 F.3d 841 (2d Cir. 1997) ? Wikipedia case summary",

"url": "https://en.wikipedia.org/wiki/National_Basketball_Ass%27n_v._Motorola,_Inc."

```

},
{
  "name": "17 U.S.C. § 102 ? BitLaw / U.S. Code",
  "url": "https://www.bitlaw.com/source/17usc/102.html"
},
{
  "name": "17 U.S.C. § 101 Definitions ? Legal Information Institute (Cornell)",
  "url": "https://www.law.cornell.edu/uscode/text/17/101"
},
{
  "name": "Sega Enterprises Ltd. v. Accolade, Inc., 977 F.2d 1510 (9th Cir. 1992) ? BitLaw",
  "url": "https://www.bitlaw.com/source/cases/copyright/Sega-Accolade.html"
},
{
  "name": "Authors Guild v. Google, Inc., 804 F.3d 202 (2d Cir. 2015) ? Justia",
  "url": "https://law.justia.com/cases/federal/appellate-
courts/ca2/13-4829/13-4829-2015-10-16.html"
},
{
  "name": "Google LLC v. Oracle Am., Inc., 141 S. Ct. 1183 (2021) ? U.S. Copyright Office
Fair Use Index summary",
  "url": "https://www.copyright.gov/fair-use/summaries/google-llc-oracle-am-inc-2021.pdf"
},
{
  "name": "Thomson Reuters Enterprise Centre GmbH v. Ross Intelligence Inc. (D. Del. 2025) ?
full opinion (BitLaw)",
  "url": "https://www.bitlaw.com/source/cases/copyright/Thomson-Reuters-Ross.html"
},
{
  "name": "Perkins Coie ? Fair Use Defense Failed in Thomson Reuters v. Ross",
  "url": "https://perkinscoie.com/insights/update/fair-use-defense-failed-thomson-reuters-v-
ross-jury-still-out-generative-ai"
},
{
  "name": "ArentFox Schiff ? Landmark Ruling on AI Copyright: Bartz v. Anthropic",
  "url": "https://www.afslaw.com/perspectives/alerts/landmark-ruling-ai-copyright-fair-use-
vs-infringement-bartz-v-anthropic"
},
{
  "name": "Authors Guild ? Bartz v. Anthropic Settlement (~$1.5B; piracy-based liability)",
  "url": "https://authorsguild.org/advocacy/artificial-intelligence/what-authors-need-to-
know-about-the-anthropic-settlement/"
},
{
  "name": "Mayer Brown ? U.S. Copyright Office (May 2025, Part 3) on AI training and fair
use",
  "url": "https://www.mayerbrown.com/en/insights/publications/2025/05/united-states-
copyright-office-weighs-in-on-fair-use-defense-for-generative-ai-training"
},
{
  "name": "ProCD, Inc. v. Zeidenberg, 86 F.3d 1447 (7th Cir. 1996) ? Wikipedia",
  "url": "https://en.wikipedia.org/wiki/ProCD,_Inc._v._Zeidenberg"
},
{
  "name": "EUR-Lex ? Case C-203/02 British Horseracing Board v William Hill (9 Nov 2004)",
  "url": "https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex%3A62002CJ0203"
},
{
  "name": "Wolters Kluwer / Kluwer Copyright Blog ? Database Directive sports-data cases",
  "url": "https://legalblogs.wolterskluwer.com/copyright-blog/databases-sui-generis-
protection-and-copyright-protection/"
},
{
  "name": "EUR-Lex ? Case C-604/10 Football Dataco v Yahoo (1 Mar 2012)",
  "url": "https://eur-lex.europa.eu/legal-content/EN/ALL/?uri=CELEX:62010CJ0604"
}

```



```

},
{
  "name": "ipcuria ? Case C-173/11 Football Dataco v Sportradar (18 Oct 2012)",
  "url": "https://ipcuria.eu/case?reference=C-173/11"
},
{
  "name": "ipcuria ? Case C-762/19 CV-Online Latvia v Melons (3 Jun 2021)",
  "url": "https://ipcuria.eu/case?reference=C-762%2F19"
},
{
  "name": "Bird & Bird ? CV-Online Latvia: CJEU complicates enforcement of database rights",
  "url": "https://www.twobirds.com/en/insights/2021/uk/cv-online-latvia-cjeu-complicates-the-enforcement-of-database-rights"
},
{
  "name": "Simkins ? Database rights back on the sport radar (Floyd J: live match events 'obtained')",
  "url": "https://www.simkins.com/news/database-rights-back-on-the-sport-radar"
},
{
  "name": "LexisNexis UK ? Legal protection of databases in the UK (post-Brexit assimilated sui generis right)",
  "url": "https://www.lexisnexis.com/en-gb/legal/guidance/legal-protection-of-databases-in-the-uk"
},
{
  "name": "EUR-Lex ? Case C-30/14 Ryanair v PR Aviation (15 Jan 2015)",
  "url": "https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:62014CJ0030"
},
{
  "name": "EBC v D.B. Modak (WIPO Lex / Supreme Court of India)",
  "url": "https://www.wipo.int/wipolex/en/judgments/details/2840"
},
{
  "name": "Ikigai Law ? Legality of data scraping in India (OLX v Padawan; IT Act s.43; ToS; database copyright)",
  "url": "https://www.ikigailaw.com/article/263/legality-of-data-scraping-in-india"
},
{
  "name": "IAPP ? Scraping public data in India: innovation enabler or privacy threat? (Govt s.43 position; DPDP s.3(c)(ii))",
  "url": "https://iapp.org/news/a/scraping-public-data-in-india-innovation-enabler-or-privacy-threat-"
},
{
  "name": "CaseMine ? Burlington Home Shopping v Rajnish Chibber (database as literary work)",
  "url": "https://www.casemine.com/commentary/in/recognition-of-database-compilation-as-a-literary-work:-analysis-of-burlington-home-shopping-pvt-ltd.-v.-rajnish-chibber-&-anr./view"
},
{
  "name": "CaseMine ? V. Govindan v E.M. Gopalakrishna Kone (compilation copyright)",
  "url": "https://www.casemine.com/commentary/in/establishing-copyright-infringement-in-dictionary-compilations:-v.-govindan-v.-e.m-gopalakrishna-kone/view"
},
{
  "name": "S.S. Rana & Co. ? Data scraping issues in India (IT Act s.43/s.66 criminal exposure)",
  "url": "https://ssrana.in/articles/data-scraping-issues-india/"
},
{
  "name": "The New Publishing Standard ? Delhi HC reserves judgment in ANI v OpenAI (April 2026)",
  "url": "https://thenewpublishingstandard.com/2026/04/01/ani-vs-openai-delhi-high-court-judgment-ai-copyright-india/"
}

```

```

    },
    {
      "name": "DPDPA s.9 ? children's data: verifiable consent + behavioural-tracking/targeted-ads prohibition",
      "url": "https://www.dpdpa.com/dpdpa2023/chapter-2/section9.html"
    },
    {
      "name": "DPDP Rules 2025, Rule 10 ? verifiable parental consent",
      "url": "https://www.dpdpa.com/dpdparules/rule10.html"
    },
    {
      "name": "DPDP Rules 2025, Rule 12 ? exemptions from s.9(1)/s.9(3) for Fourth Schedule classes/purposes",
      "url": "https://www.dpdpa.com/dpdparules/rule12.html"
    },
    {
      "name": "Medianama ? New Data Rules Allow Child Tracking for Safety Without Consent (Rule 12 carve-out)",
      "url": "https://www.medianama.com/2025/11/223-dpdp-rules-real-time-child-tracking-without-consent/"
    },
    {
      "name": "CyberPeace ? Prohibition of behavioural tracking and targeted advertising for children under DPDP s.9(3)",
      "url": "https://cyberpeace.org/resources/blogs/prohibition-of-behavioral-tracking-and-targeted-advertising-for-children-under-the-dpdp-act-2023"
    },
    {
      "name": "S.S. Rana & Co. ? Protecting trade secrets in India (Contract Act 1872, breach of confidence, IT Act s.72)",
      "url": "https://ssrana.in/articles/protecting-trade-secrets-india/"
    },
    {
      "name": "K Singhania & Co. ? Regulation of biometric data under the DPDP Act (facial data = personal data)",
      "url": "https://ksandk.com/data-protection-and-data-privacy/regulation-of-biometric-data-under-the-dpdp-act/"
    },
    {
      "name": "Van Buren v. United States, No. 19-783 (U.S. June 3, 2021) ? slip opinion",
      "url": "https://www.supremecourt.gov/opinions/20pdf/19-783_k53l.pdf"
    },
    {
      "name": "Zwillgen ? Van Buren: footnote 8 reserved whether contract/policy limits ground CFAA liability",
      "url": "https://www.zwillgen.com/alternative-data/scotus-rules-cfaa-does-not-contain-purpose-based-restrictions/"
    },
    {
      "name": "Privacy World (Squire Patton Boggs) ? hiQ v. LinkedIn breach-of-contract summary judgment",
      "url": "https://www.privacyworld.blog/2022/11/federal-court-rules-in-favor-of-linkedins-breach-of-contract-claim-after-six-years-of-cfaa-data-scraping-litigation/"
    },
    {
      "name": "Privacy World ? hiQ v. LinkedIn $500k consent judgment, injunction and destruction of scraped-derived data",
      "url": "https://www.privacyworld.blog/2022/12/linkedins-data-scraping-battle-with-hiq-labs-ends-with-proposed-judgment/"
    },
    {
      "name": "Eric Goldman / Quinn Emanuel ? Meta v. Bright Data (N.D. Cal. Jan. 23, 2024): logged-off public scraping not barred",
      "url": "https://www.quinnemanuel.com/the-firm/news-events/client-alert-meta-v-bright-data-significant-decision-for-web-scraping-industry/"
    }
  ]

```

```

    },
    {
      "name": "Crowell & Moring ? browwrap vs clickwrap enforceability (notice + assent)",
      "url": "https://www.crowell.com/en/insights/client-alerts/recent-court-rulings-provide-warnings-on-the-use-of-browwrap-agreements"
    },
    {
      "name": "Hunton ? sign-in wrap enforceability: courts deny absent conspicuous notice / unambiguous assent",
      "url": "https://www.hunton.com/privacy-and-information-security-law/its-a-wrap-the-latest-from-the-ninth-circuit-on-sign-in-wrap-agreements"
    },
    {
      "name": "YouTube Terms of Service ? Permissions and Restrictions (automated access; harvest faces; redistribution)",
      "url": "https://www.youtube.com/static?template=terms"
    },
    {
      "name": "18 U.S.C. § 1839 ? Trade Secret Definitions (Cornell LII)",
      "url": "https://www.law.cornell.edu/uscode/text/18/1839"
    },
    {
      "name": "Gowling WLG ? Trade secrets in professional sports",
      "url": "https://gowlingwlg.com/en/insights-resources/articles/2023/trade-secrets-in-professional-sports"
    },
    {
      "name": "Lexology ? In-venue filming of live sporting events: ticket as contract; venue/broadcaster rights",
      "url": "https://www.lexology.com/library/detail.aspx?g=09fe6573-daf6-4fff-bcf0-b9c6ccf41792"
    },
    {
      "name": "GDPR Recital 26 (official text, gdpr-info.eu)",
      "url": "https://gdpr-info.eu/recitals/no-26/"
    },
    {
      "name": "gdpr-info.eu ? Personal Data (Art.4(1)); WP136 'relates to' test",
      "url": "https://gdpr-info.eu/issues/personal-data/"
    },
    {
      "name": "gdpr-info.eu ? Art.9 GDPR (special-category biometric data); ICO biometric guidance",
      "url": "https://gdpr-info.eu/art-9-gdpr/"
    },
    {
      "name": "gdpr-info.eu ? Art.8 GDPR (children's consent)",
      "url": "https://gdpr-info.eu/art-8-gdpr/"
    },
    {
      "name": "Fox Rothschild ? EDPB Guidelines 3/2019 on video devices (Art.9 only on unique identification)",
      "url": "https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-issues-final-guidance"
    },
    {
      "name": "Weiss et al., Sensors 2025 (PMC) ? Privacy Beyond the Face: gait identity survives pose anonymization",
      "url": "https://pmc.ncbi.nlm.nih.gov/articles/PMC12788357/"
    },
    {
      "name": "Bird & Bird ? CJEU C-582/14 Breyer: relative approach to identifiability",
      "url": "https://www.twobirds.com/en/insights/2016/global/cjeu-decision-on-dynamic-ip-addresses-touches-fundamental-dp-law-questions"
    },
  ],

```

```

{
  "name": "Article 29 Working Party Opinion 05/2014 (WP216) on Anonymisation Techniques",
  "url": "https://ec.europa.eu/justice/article-29/documentation/opinion-recommendation/files/2014/wp216_en.pdf"
},
{
  "name": "Hunton ? EDPB Guidelines 01/2025 on pseudonymisation (context-dependent identifiability)",
  "url": "https://www.hunton.com/insights/publications/edpb-advises-on-pseudonymisation-for-gdpr-compliance"
},
{
  "name": "Clifford Chance ? Pseudonymized data after EDPS v SRB (CJEU C-413/23 P, 4 Sept 2025)",
  "url": "https://www.cliffordchance.com/insights/resources/blogs/talking-tech/en/articles/2025/09/pseudonymized-data-after-edps-v-srb.html"
},
{
  "name": "Skadden ? EU Court Clarifies When Pseudonymised Data Is Not Personal Data (EDPS v SRB)",
  "url": "https://www.skadden.com/insights/publications/2025/11/in-a-landmark-decision-eu-court-clarifies"
},
{
  "name": "ICO ? How do we ensure anonymisation is effective? (motivated-intruder test)",
  "url": "https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-sharing/anonymisation/how-do-we-ensure-anonymisation-is-effective/"
},
{
  "name": "NIST CSRC ? SP 800-188 De-Identifying Government Datasets (release-model framing)",
  "url": "https://csrc.nist.gov/News/2023/nist-publishes-sp-800-188"
},
{
  "name": "Sweeney, k-Anonymity: A Model for Protecting Privacy (EPIC mirror)",
  "url": "https://epic.org/wp-content/uploads/privacy/reidentification/Sweeney_Article.pdf"
},
{
  "name": "de Montjoye et al., Unique in the Crowd (Scientific Reports 2013) ? 4 points -> 95%",
  "url": "https://dspace.mit.edu/bitstream/handle/1721.1/88233/de%20Montjoye-2013-Unique%20in%20the%20crowd.pdf?sequence=1&isAllowed=y"
},
{
  "name": "OVIC ? The Limitations of De-Identification (data environment / external linkage; forward-looking)",
  "url": "https://ovic.vic.gov.au/privacy/resources-for-organisations/the-limitations-of-de-identification-protecting-unit-record-level-personal-information/"
},
{
  "name": "Trajectory-Based Spatiotemporal Entity Linking (arXiv 2010.01516)",
  "url": "https://arxiv.org/pdf/2010.01516"
}
]
}

```

GROUNDING:

CODEBASE FACTS (verified by the lead before this run):

- eval_baselines/ is a TRACKED (committed) dir of JSON baseline snapshots (regression_baseline.json, heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json, experiment_leaderboard.json). Precedent: committed structured regression files already live in git.
- backend/eval/regression.py loads a committed baseline, scores DB-stored predictions vs ground-truth intervals, flags P/R drop beyond tolerance, with incommensurability guards (IoU/detector/GT-hash). Predictions come from the DB (needs a pipeline run on the video) -> NOT

video-free on the prediction side yet.

- backend/eval/fusion_golden.py extracts *_golden_features.json from VIDEO+trajectory CSV+labels (needs GPU/WSL). backend/eval/fusion_compare.py & backend/eval/ablation.py RE-SCORE those JSON features on pure CPU with NO video (LOVO deltaF1, bootstrap CI, sign test). This is the EXISTING VIDEO-FREE seam.

- docs/GOLDEN_DATA_SHARING.md (#175): *_golden_features.json are deliberately kept PRIVATE on OneDrive, OUT of git, because they derive from a 'Proprietary corpus.' Existing skipif litmus tests (tests/test_ablation.py::test_litmus_reproduces_gate_a, tests/test_fusion_compare.py) run ONLY if those features are present in output/ -> currently skipped everywhere.

- Data contracts: trajectory CSV columns (Frame,Visibility,X,Y); candidate/final rally segments = (start,end) time intervals; multi-signal fusion uses shuttle + person + audio + court-bounds feature columns (weak signals retained, default-OFF). video_id = MD5 of the source file (backend/utils/hashing.py) ? i.e. using it as a public key would FINGERPRINT the source file.

- Guardrails: commercial-clean licensing MIT/Apache/BSD only; paid LLMs never a training source; EXCLUDE MINORS from the training pool; gait/biometrics strictly FUTURE (GDPR Art 9). Owner is Bangalore/India-based; aims at commercialization and possibly co-authoring a research paper.

- A SEPARATE workflow is concurrently designing the general video-free fixture mechanics. THIS effort focuses on (a) the LEGAL/IP/privacy status of committing DERIVED structured data PUBLICLY, and (b) a non-attributability + encryption ('secret file') architecture.

GOAL: let an external contributor on another clone run the rally-segmentation regression suite from COMMITTED structured fixtures (no video present), such that the fixtures (i) cannot be attributed to a specific source video/match/person and (ii) are useless to a public user in isolation.

KEY CONSTRAINT ALREADY ESTABLISHED BY THE LEAD: a contributor who RUNS the tests necessarily can read the plaintext fixtures + any key on their machine (DRM/analog-hole limit). So secrecy-from-contributor is impossible; the achievable goal is NON-ATTRIBUTABILITY + uselessness-in-isolation, plus encryption only as defense against the NO-KEY public.

2. assistant (2026-06-16T03:05:58.729Z)

I'll analyze the codebase to ground my design in the actual existing seams, then design the encryption/secret-file layer with the tiered model. Let me start by reading the key files referenced in the grounding.

3. user (2026-06-16T03:05:59.692Z)

```
1      # Golden / GPU-Extractor Data Sharing ? GPU box ? dev sessions
2
3      **Status:** convention + plan (2026-06-15). Tracked in **#175**.
4      **Purpose:** a single cloud-synced location for the GPU-extracted golden artifacts
(trajecory CSVs, `*_golden_features.json`, manifests, baseline snapshots) so the **GPU box**
and **CPU dev/agent sessions** share the same files ? without committing large data to git.
5
6      ## Why this exists
7      The rally-quality measurement is split across two machines by capability:
8      - **`backend/eval/fusion_golden.py`** (shuttle+person feature **extraction**) needs the
**videos**, the **trajectory CSVs**, the **`.rallies.csv` golden labels**, and a **GPU/WSL**
stack ? only the GPU box can run it.
9      - **`backend/eval/fusion_compare.py`** / **`backend/eval/ablation.py`** (the LOVO
**litmus**: ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the
small `*_golden_features.json` ? any session (incl. a CPU dev container) can run them.
10
11     Today those artifacts sit in gitignored `output/` (per-machine) and scattered local
folders (`...\Annotation Setup\Collect\Trajectories\`, `...\Golden Labelled\`). Nothing syncs
them between machines, so a CPU session cannot see the GPU box's extraction. This wires that gap
with OneDrive.
12
13     ## Canonical shared folder
14     `%USERPROFILE%\OneDrive\rally-annotator\golden-data\` ?
`C:\Users\avidu\OneDrive\rally-annotator\golden-data\`
15
16     OneDrive is present + signed-in on both the GPU box and the dev machine under the same
account, so it auto-syncs. Layout (date/size-tagged so a re-extraction is a new folder, never an
overwrite):
17
```

```

18     ``
19     golden-data/
20     README.md
21     features/<corpus-tag>/      *_golden_features.json      (small ? the key shared
artifact)
22     trajectories/<corpus-tag>/  *_predict.csv / *_wasb.csv (medium ? reusable across
runs)
23     manifests/                  *_lovo_manifest.json
24     baselines/                  leaderboard / regression snapshots (or keep tracked in
eval_baselines/)
25     ``
26
27     `<corpus-tag>` = e.g. `2026-06-15-n6` (date + corpus size). **Do NOT** put raw videos
here (tens of GB) ? keep those local to the GPU box.
28
29     ## The GPU ? CPU workflow
30     1. **GPU box** (videos + CSVs + labels present):
31     ``
32     python -m backend.eval.fusion_golden --manifest <manifest> \
33     --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-
data\features\2026-06-15-n6"
34     ``
35     (or extract to local `output/` then copy the `*_golden_features.json` into the shared
folder).
36     2. OneDrive syncs the JSONs to the dev machine automatically (small files, seconds).
37     3. **CPU dev/agent session** points the analysis at the shared folder:
38     ``
39     python -m backend.eval.fusion_compare --features-dir "...golden-
data\features\2026-06-15-n6" --record
40     python -m backend.eval.ablation      --features-dir "...golden-
data\features\2026-06-15-n6" --granularity group
41     ``
42     ? real person-feature ?F1 / CI / sign test. Copy the JSONs into the repo's `output/`
to also light up the `skipif` golden-litmus tests
(`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`).
43
44     ## Optional code ergonomics (in #175, not required)
45     - A `RALLY_GOLDEN_DIR` env var (default `output/`) honored by `fusion_golden` /
`fusion_compare` / `ablation` so neither machine hardcodes the path. (The CLI `--out-dir` /
`--features-dir` flags already make this a convention; the env var is just ergonomics.)
46     - A tiny `scripts/sync_golden.py` to copy `output/*_golden_features.json` ? the shared
folder.
47
48     ## Guardrails
49     - The shared folder lives **outside the repo** ? never committed. Keep `output/`
gitignored and do **not** symlink the shared folder into a tracked path.
50     - The features derive from the **Proprietary** corpus ? keep the OneDrive folder
**private** (no public share links).
51
52     See also: `docs/NEXT_STEPS.md` (the "Next-session pickup" anchor),
`docs/MULTI_SIGNAL_FUSION_PLAN.md` (P3), `docs/TRACKNET_WSL_SETUP.md` (the GPU/WSL detector
setup).
53

```

4. user (2026-06-16T03:06:02.577Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's

```

```

6      designed seam): the person features are extra columns the learned LogisticRegression
weights,
7      so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9      Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58                threshold=threshold)
59         fusion = experiment.Variant(name="shuttle_person_fusion",
60                feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61                threshold=threshold)
62         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64

```

```

65     def format_result(res: Dict[str, Any], iou: float) -> str:
66         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
67         ds = delta_stats(res)
68         st = ds["sign_test"]
69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
74             f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
75             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
76             f"| CI clears 0: {ds['ci_excludes_zero']}",
77             "  per-video ?F1:",
78         ]
79         for pv in res["per_video_delta"]:
80             lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
81         return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

5. user (2026-06-16T03:06:02.606Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7     ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8     predictions  ? the production pipeline's stored DB rows    [backend.eval.harness]
9     comparison   ? interval matching + P/R/F1                  [backend.eval.metrics]
10    baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an

```



```

15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"

```

```

78
79
80 def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                  f1: float, path: str = DEFAULT_BASELINE_PATH,
82                  iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                  gt_fingerprint: Optional[str] = None) -> None:
84     """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86     Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87     later run computed under different settings (or against different labels) is
88     as incommensurable rather than silently compared.
89     """
90     baselines = load_baselines(path)
91     baselines[_baseline_key(video_id, stage)] = {
92         "video_id": video_id, "stage": stage,
93         "precision": precision, "recall": recall, "f1": f1,
94         "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95     }
96     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97     with open(path, "w") as f:
98         json.dump(baselines, f, indent=2, sort_keys=True)
99     logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100 _baseline_key(video_id, stage), precision, recall)
101
102 # ----- the runner
103
104 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105            stage: str = "final", iou_threshold: float = 0.5,
106            detector: Optional[str] = None,
107            tolerance: float = DEFAULT_TOLERANCE,
108            baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109     """Score stored predictions for one video against ground truth and compare to
110     baseline.
111
112     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
113     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
114     precision OR recall falls more than `tolerance` below the snapshotted baseline.
115     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
116     caller can choose to snapshot the current numbers via `save_baseline`.
117     """
118     preds = get_predictions(db, video_id, stage, detector=detector)
119     s = metrics.score(preds, ground_truth, iou_threshold)
120     fp = gt_hash(ground_truth)
121
122     baselines = load_baselines(baseline_path)
123     b = baselines.get(_baseline_key(video_id, stage))
124
125     reasons: List[str] = []
126     regressed = False
127     has_baseline = b is not None
128     base_p = base_r = None
129     if b is not None:
130         base_p, base_r = b["precision"], b["recall"]
131         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
132         # against different labels is not a valid reference ? flag it instead of
133         # trusting it.
134         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
135         b.get("gt_hash")
136         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
137             regressed = True
138             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
139 (incommensurable)")

```

```

136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
157         )
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                             tier: str = "file", stage: str = "final",
162                             iou_threshold: float = 0.5,
163                             detector: Optional[str] = None,
164                             tolerance: float = DEFAULT_TOLERANCE,
165                             baseline_path: str = DEFAULT_BASELINE_PATH,
166                             guideline: Optional[str] = None,
167                             sport: str = "badminton",
168                             provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + " ".join(f"\n    - {why}" for why in r.reasons)
189

```

6. assistant (2026-06-16T03:06:05.235Z)

Now let me look at the actual feature JSON schema and related eval files to ground the design precisely.

7. user (2026-06-16T03:06:06.221Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31      from backend.utils.run_telemetry import RunTelemetry
32
33      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39      def _derive_video_path(spec: Dict[str, Any]) -> str:
40          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41          golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42          if spec.get("video_path"):
43              return str(spec["video_path"])
44          labels = str(spec["labels"])
45          if labels.endswith(".rallies.csv"):
46              return labels[: -len(".rallies.csv")] + ".MP4"
47          raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50      def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                      frame_skip: int = 3, iou: float = 0.5,
52                      log_every_sec: float = 15.0,
53                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
```

```

54         """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56         Returns ``{name, fps, frame_width,
candidates:[{start,end,shuttle{},person{},label}], gts}``.
57         `detector` (anything with ``detections_over_windows``) is injectable for GPU-free
tests;
58         when None a real `PersonDetector` is built (loads torchvision on first use).
59         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60         from the detection progress callback so a hard kill leaves a forensic runlog.
61         """
62         fps, fw = float(spec["fps"]), float(spec["frame_width"])
63         traj = str(spec["trajectory"])
64         points = TrackNetRunner.parse_trajectory_csv(traj)
65         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66         gts = load_golden_gts(str(spec["labels"]))
67         labels = label_candidates(intervals, gts, iou)
68         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69         # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70         axis, net_px, _span = rally_gate.estimate_play_axis(points)
71         video_path = _derive_video_path(spec)
72
73         if detector is None:
74             from backend.pipeline.detectors.person_detector import PersonDetector
75             detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79         n = len(intervals)
80         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81         print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82         t0 = time.monotonic()
83         state = {"last": t0}
84
85         def _progress(done: int, total: int) -> None:
86             if telemetry is not None:
87                 telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                       f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96                 state["last"] = now
97
98         per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                         progress_cb=_progress)
100
101         candidates: List[Dict[str, Any]] = []
102         for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103             det = per_window[i]
104             det_fw = det.get("frame_width") or fw
105             det_fh = det.get("frame_height") or 0.0
106             # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107             # normalisation fusion_features uses (x/width, y/height).
108             if axis == "x":
109                 net_norm = net_px / det_fw if det_fw > 0 else 0.5

```

```

110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112             sf, ef = win_frames[i]
113             shuttle_in = [p for p in points if sf <= p.frame <= ef]
114             person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                                court_polygon=None, net=(axis, net_norm))
116             candidates.append({
117                 "start": float(s), "end": float(e),
118                 "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119                 "person": {k: float(v) for k, v in person.items()},
120                 "label": int(labels[i]),
121             })
122         return {
123             "name": spec["name"], "fps": fps, "frame_width": fw,
124             "candidates": candidates,
125             "gts": [[float(a), float(b)] for a, b in gts],
126         }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
136             # row/frame).
137             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
138                                                if os.path.isfile(s.get("trajectory", ""))
139                                                else 0))
140         os.makedirs(out_dir, exist_ok=True)
141         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
142         # keep a tiny
143         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
144         # must never
145         # break extraction, so guard the build and fall back to no telemetry on any error.
146         telem: Optional[RunTelemetry]
147         try:
148             telem = RunTelemetry(os.path.join(out_dir, "run-telemetry.jsonl"),
149                                label="fusion_golden")
150         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
151             telem = None
152         written: List[str] = []
153         for vi, spec in enumerate(specs, 1):
154             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
155             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
156             # prior
157             # run left off instead of re-detecting). --fresh forces a redo.
158             if not fresh and os.path.isfile(out):
159                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
160                 extracted "
161                        f"-> {out} (use --fresh to redo)", flush=True)
162                 written.append(out)
163                 continue
164             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
165                  flush=True)
166             t0 = time.monotonic()
167             if telem is not None:
168                 telem.event("video_start", spec["name"])
169             try:
170                 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
171             except Exception as e: # noqa: BLE001 - record machine state at death, then re-
172                 raise

```

```

164         if telem is not None:
165             telem.event("error", spec["name"], err=str(e))
166         raise
167     n_pos = sum(c["label"] for c in data["candidates"])
168     with open(out, "w", encoding="utf-8") as f:
169         json.dump(data, f, indent=2)
170     if telem is not None:
171         telem.event("video_done", spec["name"],
172                     windows=len(data["candidates"]), positives=n_pos)
173     print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
174          f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175          f"-> {out}", flush=True)
176     written.append(out)
177     return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

8. user (2026-06-16T03:06:06.805Z)

tests\test_ablation.py

9. user (2026-06-16T03:06:06.977Z)

tests\test_fusion_compare.py

10. assistant (2026-06-16T03:06:10.014Z)

Let me read the test files to understand the existing skipif pattern, and check the eval_baselines dir + .gitignore for the committed-precedent context.

11. user (2026-06-16T03:06:10.310Z)

```

1     """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3     The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4     non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5     report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6     """
7     import json

```

```

8     import os
9     from types import SimpleNamespace
10
11     import pytest
12
13     from backend.eval import fusion_compare
14     from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17     def _write_video(d, name):
18         """A video whose shuttle features are identical for pos/neg (no signal) but whose
19         players_in_court separates rallies (4) from junk (0)."""
20         cand = []
21         for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
22         51.0), 0)]:
23             shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES} # constant
24             # non-separating
25             person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
26             person["players_in_court"] = 4.0 if pos else 0.0 # the
27             # separating signal
28             person["in_court_ratio"] = 1.0 if pos else 0.0
29             cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
30             "label": pos})
31             data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
32             "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
33             with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
34             f:
35                 json.dump(data, f)
36
37
38     def test_fusion_compare_reports_person_driven_lift(tmp_path):
39         d = str(tmp_path)
40         for k in range(6):
41             _write_video(d, f"vid{k}")
42         videos = fusion_compare.load_videos(d)
43         assert len(videos) == 6
44         res = fusion_compare.compare(videos, iou=0.5)
45         assert res["num_videos"] == 6
46         assert res["variant_b"]["honest_lovo"] is True # LOVO-honest learned
47         # variant
48         # Fusion (with the separating person feature) must beat shuttle-only (no signal).
49         assert res["delta"]["f1"] > 0.0
50         assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
51         # format_result must not crash and must mention both variants.
52         out = fusion_compare.format_result(res, 0.5)
53         assert "shuttle-only" in out and "+person" in out
54
55     def test_fusion_compare_delta_stats_and_ci(tmp_path):
56         """delta_stats yields a paired bootstrap CI + sign test; format_result reports
57         them."""
58         d = str(tmp_path)
59         for k in range(6):
60             _write_video(d, f"vid{k}")
61         res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
62         ds = fusion_compare.delta_stats(res)
63         assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
64         "ci_excludes_zero"}
65         assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9 # CI
66         # brackets mean (±fp)
67         assert ds["sign_test"]["wins"] == 6 # fusion wins on every video (person
68         # feature separates)
69         assert ds["ci_excludes_zero"] is True
70         out = fusion_compare.format_result(res, 0.5)
71         assert "CI" in out and "sign test" in out

```



```

63
64
65 def test_fusion_compare_needs_two_videos(tmp_path):
66     _write_video(str(tmp_path), "only")
67     videos = fusion_compare.load_videos(str(tmp_path))
68     assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO
69
70
71 # ----- extractor assembly
72
73 def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74     from backend.eval import fusion_golden
75     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
76
77     intervals = [(0.0, 10.0), (40.0, 41.0)]
78     x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
79     points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))]
80         for i in range(10)]
81
82     monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
83                         lambda traj, fps, fw: (intervals, x_shuttle))
84     monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
traj: points))
85     monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
86
87     class _StubDetector:
88         def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
89             return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
90                        "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
91                     for (sf, ef) in windows]
92
93     spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
94            "labels": "foo.rallies.csv"}
95     data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
iou=0.5)
96
97     assert data["name"] == "t"
98     assert data["gts"] == [(0.0, 10.0)]
99     assert len(data["candidates"]) == 2
100    c0 = data["candidates"][0]
101    assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
102    assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
103    assert c0["label"] == 1 # (0,10) matches the gt -> positive
104    assert data["candidates"][1]["label"] == 0 # (40,41) -> negative
105    assert all(isinstance(v, float) for v in c0["person"].values())
106
107
108 def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
109     """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
all."""
110     from backend.eval import fusion_golden
111     out_dir = str(tmp_path / "out")
112     os.makedirs(out_dir)
113     manifest = [
114         {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "a.rallies.csv"},
115         {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "b.rallies.csv"},
116     ]
117     manifest_path = str(tmp_path / "m.json")
118     with open(manifest_path, "w", encoding="utf-8") as f:
119         json.dump(manifest, f)
120     # a prior run already produced alpha's JSON
121     with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",

```

```

encoding="utf-8") as f:
122     json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}, f)
123
124     extracted = []
125
126     def _stub(spec, **kw):
127         extracted.append(spec["name"])
128         return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
129
130     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
131
132     fusion_golden.run(manifest_path, out_dir, fresh=False)
133     assert extracted == ["beta"] # alpha skipped (already extracted)
134
135     extracted.clear()
136     fusion_golden.run(manifest_path, out_dir, fresh=True)
137     assert extracted == ["alpha", "beta"] # --fresh redoes both
138
139
140     def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
141         """--smallest-first orders by trajectory size; default keeps manifest order."""
142         from backend.eval import fusion_golden
143         out_dir = str(tmp_path / "out")
144         os.makedirs(out_dir)
145         (tmp_path / "big.csv").write_text("x" * 2000)
146         (tmp_path / "small.csv").write_text("x" * 20)
147         manifest = [
148             {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
149             {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "s.rallies.csv"},
150         ]
151         mp = str(tmp_path / "m.json")
152         with open(mp, "w", encoding="utf-8") as f:
153             json.dump(manifest, f)
154
155         order = []
156
157         def _stub(spec, **kw):
158             order.append(spec["name"])
159             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
160
161         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
162
163         fusion_golden.run(mp, out_dir, smallest_first=True)
164         assert order == ["small", "big"] # smaller trajectory first
165
166         order.clear()
167         fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
168         assert order == ["big", "small"] # manifest order preserved
169
170
171     def test_derive_video_path():
172         from backend.eval.fusion_golden import _derive_video_path
173         assert _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"}) == "X/Adarsh
and Avi.MP4"
174         assert _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"}) ==
"a.mp4"
175         with pytest.raises(ValueError):
176             _derive_video_path({"name": "n", "labels": "weird.csv"})
177

```

12. user (2026-06-16T03:06:10.846Z)

```
1      """GPU-free tests for the LOSO/LOGO ablation runner (`backend.eval.ablation`).
2
3      Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5      - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise
signal is
6      *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns /
zeroes
7      the gate knob without mutating ``full``.
8      - the structural-guard exemption ? the SAME signal marked ``structural=True`` is
reported
9      (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no
lift".
10     - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
stronger
11     than the stats support (the no-over-claim contract).
12     - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
13     - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
14     reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
15     cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16     """
17     from __future__ import annotations
18
19     import glob
20     import json
21     import os
22
23     import pytest
24
25     from backend.eval import ablation
26     from backend.eval.ablation import (
27         AblationConfig,
28         AblationReport,
29         Signal,
30         ablate_one,
31         run_ablation,
32     )
33     from backend.eval.experiment import Variant, VideoCandidates
34
35
36     # ----- synthetic
fixtures
37
38
39     def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
40         """`n` videos with a ``net_crossings`` gate signal.
41
42         ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
``min_crossings=2`` gate
43         perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
44         has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
45         videos: list[VideoCandidates] = []
46         for k in range(n):
47             intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
48             junk_val = 5.0 if not separating else 0.0
49             feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
50             gts = [(0.0, 10.0), (20.0, 30.0)]
51             videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
```

```

52         return videos
53
54
55     def _learned_videos(n: int = 6) -> list[VideoCandidates]:
56         """`n` videos where a PERSON feature (`players_in_court`) cleanly separates
rallies from
57         junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
58         ablation must show person as the load-bearing group and the others as washes ? a
deterministic
59         LOGO/LOSO correctness fixture (mirrors `test_fusion_compare`'s construction)."""
60         videos: list[VideoCandidates] = []
61         for k in range(n):
62             intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
63             feats: dict[str, list[float]] = {}
64             for fn in ablation.SHUTTLE_COLUMNS:
65                 feats[fn] = [1.0, 1.0, 1.0, 1.0] # constant ? non-
separating
66             for fn in ablation.PERSON_COLUMNS:
67                 feats[fn] = [0.0, 0.0, 0.0, 0.0]
68             feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0] # the separating signal
69             feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
70             for fn in ablation.AUDIO_COLUMNS:
71                 feats[fn] = [0.0, 0.0, 0.0, 0.0]
72             gts = [(0.0, 10.0), (20.0, 30.0)]
73             videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
74         return videos
75
76
77     # ----- LOSO
correctness
78
79
80     def test_separating_gate_earns_keep():
81         """A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing
it
82         drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""
83         videos = _gate_videos(6, separating=True)
84         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
85         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
86                               granularity="signal")
87         row = ablate_one(videos, cfg, sig)
88         assert row.delta_f1 > 0.0
89         assert row.ci_excludes_zero is True
90         assert row.sign_wins == 6 and row.sign_losses == 0
91         assert row.sign_p_value <= 0.05
92         assert row.verdict == ablation.EARNS_KEEP
93         assert row.structural_protected is False
94         # C4: the gate cuts over-segmentation (more junk segments without it).
95         assert row.seg_ratio_full < row.seg_ratio_ablated
96
97
98     def test_noop_gate_is_inert():
99         """A gate whose threshold removes nothing (all windows pass) is a wash ? INERT,
never a
100         stronger claim. Honest: a single wash at n=6 is a candidate, not 'proven
useless'."""
101         videos = _gate_videos(6, separating=False)
102         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
103         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
104                               granularity="signal")
105         row = ablate_one(videos, cfg, sig)
106         assert abs(row.delta_f1) < 1e-9
107         assert row.ci_excludes_zero is False

```

```

108         assert row.verdict == ablation.INERT
109
110
111     def test_ablated_variant_does_not_mutate_full():
112         """Building the ablated variant must derive a new Variant, never mutate ``full`` in
place."""
113         full = Variant(name="full", feature_names=["a", "b", "c"], min_crossings=2)
114         before = full.to_dict()
115         ablated = ablation._ablated_variant(full, ("b",), "min_crossings")
116         assert ablated.feature_names == ["a", "c"]
117         assert ablated.min_crossings == 0
118         assert full.to_dict() == before # full untouched
119         assert full.feature_names == ["a", "b", "c"]
120
121
122     def test_unknown_gate_knob_raises():
123         full = Variant(name="full", feature_names=["a"])
124         with pytest.raises(ValueError):
125             ablation._ablated_variant(full, (), "min_bogus")
126
127
128     # ----- LOGO
correctness
129
130
131     def test_logo_collapses_groups():
132         """LOGO (``granularity='group'``) ablates a whole producer's columns at once: the
default
133         5-signal LOSO inventory collapses to 4 producer groups, and the union of columns is
dropped."""
134         groups = ablation._collapse_to_groups(ablation.default_signals())
135         names = [g.name for g in groups]
136         assert names == ["shuttle", "gate_a", "person", "audio"]
137         shuttle = next(g for g in groups if g.name == "shuttle")
138         # the shuttle group unions duration + the velocity block
139         assert set(shuttle.columns) == {"duration", "peak_velocity", "mean_velocity",
"active_ratio"}
140         gate = next(g for g in groups if g.name == "gate_a")
141         assert gate.gate_knob == "min_crossings"
142         assert gate.structural is True # structural flag OR-ed up
143
144
145     def test_logo_finds_separating_group():
146         """On the learned fixture where PERSON cleanly separates, the LOGO person row must
be the
147         load-bearing one (largest positive marginal ?F1) and the constant groups must be
~washes."""
148         videos = _learned_videos(6)
149         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
150                             granularity="group")
151         report = run_ablation(videos, cfg)
152         rows = {r.signal: r for r in report.rows}
153         assert set(rows) == {"shuttle", "gate_a", "person", "audio"}
154         top = report.sorted_rows()[0]
155         assert top.signal == "person"
156         assert rows["person"].delta_f1 > 0.0
157         # the constant audio group carries no information ? marginal ? 0
158         assert abs(rows["audio"].delta_f1) < 0.05
159
160
161     def test_run_ablation_both_granularities():
162         """LOSO yields one row per signal; LOGO yields one per group; both honest-LOVO on a
learned
163         full set."""

```

```

164     videos = _learned_videos(6)
165     loso = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
166                                                signals=ablation.default_signals(),
167                                                granularity="signal"))
168     logo = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
169                                                signals=ablation.default_signals(),
170                                                granularity="group"))
171     assert len(losa.rows) == len(ablation.default_signals())    # 5 LOSO rows
172     assert len(logo.rows) == 4                                # 4 producer groups
173     assert losa.honest_loso is True and logo.honest_loso is True
174
175
176     def test_run_ablation_rejects_bad_granularity():
177         videos = _learned_videos(2)
178         cfg = AblationConfig(full=ablation.learned_full_variant(),
179                             signals=ablation.default_signals(),
180                             granularity="bogus")
181         with pytest.raises(ValueError):
182             run_ablation(videos, cfg)
183
184     def test_run_ablation_needs_a_video():
185         cfg = AblationConfig(full=ablation.learned_full_variant(),
186                             signals=ablation.default_signals())
187         from backend.eval.experiment import ExperimentError
188         with pytest.raises(ExperimentError):
189             run_ablation([], cfg)
190
191     # ----- structural-guard
192 exemption
193
194     def test_structural_guard_never_demoted():
195         """A structural guard with the SAME measured ? as a non-structural one that EARNS
196 its keep
197 must report verdict ``structural_protected`` (never auto-demoted), but record the
198 honest ?."""
199         videos = _gate_videos(6, separating=True)
200         non = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
201         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
202         full = Variant(name="full", min_crossings=2)
203         cfg = AblationConfig(full=full, signals=[non], granularity="signal")
204         r_non = ablate_one(videos, cfg, non)
205         r_struct = ablate_one(videos, cfg, struct)
206         # identical honest measurement?
207         assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
208         assert (r_non.sign_wins, r_non.sign_losses) == (r_struct.sign_wins,
209 r_struct.sign_losses)
210         # ?but the verdict differs: non-structural earns keep, structural is protected.
211         assert r_non.verdict == ablation.EARNS_KEEP
212         assert r_struct.verdict == ablation.STRUCTURAL_PROTECTED
213         assert r_struct.structural_protected is True
214
215     def test_structural_guard_protected_even_when_inert():
216         """A structural guard that is a measured wash is STILL reported as
217 ``structural_protected``
218 (never flagged dead weight on 'no measured lift')."""
219         videos = _gate_videos(6, separating=False)
220         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
221         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[struct],
222                             granularity="signal")
223         row = ablate_one(videos, cfg, struct)
224         assert abs(row.delta_f1) < 1e-9                                # honestly a wash

```

```

222         assert row.verdict == ablation.STRUCTURAL_PROTECTED
223         assert row.structural_protected is True
224
225
226     def test_default_gate_a_is_structural():
227         """The shipped Gate-A signal must carry ``structural=True`` (the held-shuttle FP
guard)."""
228         sig = next(s for s in ablation.default_signals() if s.group == "gate_a")
229         assert sig.structural is True
230         assert sig.gate_knob == "min_crossings"
231
232
233     # ----- honest-
stats fields
234
235
236     def test_row_carries_full_honest_evidence():
237         """Every row ships the honest evidence: n, CI bounds, the sign test, and a verdict
in the
238         closed vocabulary (no over-claim)."""
239         videos = _learned_videos(6)
240         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
241                             granularity="group")
242         report = run_ablation(videos, cfg)
243         valid = {ablation.EARNS_KEEP, ablation.SUGGESTIVE, ablation.INERT,
244                 ablation.STRUCTURAL_PROTECTED}
245         for r in report.rows:
246             d = r.as_dict()
247             assert {"delta_f1", "ci_low", "ci_high", "ci_excludes_zero", "sign_test", "n",
248                    "verdict"} <= set(d)
249             assert d["n"] == 6
250             assert d["sign_test"]["wins"] + d["sign_test"]["losses"] <= 6
251             # CI brackets the mean ( $\pm$ fp) ? never a bare mean detached from its interval.
252             assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
253             assert r.verdict in valid
254
255
256     def test_classify_verdicts_cover_all_branches():
257         """``_classify`` maps evidence ? the closed verdict vocabulary, honestly:
258         structural always protected; CI-clears-0 + sign-agrees ? earns_keep; leans-positive
but
259         CI-spans-0 ? suggestive; a wash ? inert."""
260         # structural always protected, regardless of evidence
261         assert ablation._classify(True, True, 6, 0, 0.01, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
262         assert ablation._classify(True, False, 0, 6, 1.0, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
263         # earns keep: CI clears 0 AND sign agrees
264         assert ablation._classify(False, True, 6, 0, 0.03, 0.05) == (ablation.EARNS_KEEP,
False)
265         # suggestive: leans positive (more wins) but CI spans 0 / not significant
266         assert ablation._classify(False, False, 4, 2, 0.30, 0.05) == (ablation.SUGGESTIVE,
False)
267         # inert: not more wins than losses and CI spans 0
268         assert ablation._classify(False, False, 2, 4, 0.69, 0.05) == (ablation.INERT, False)
269         assert ablation._classify(False, False, 2, 2, 1.0, 0.05) == (ablation.INERT, False)
270
271
272     def test_no_over_claim_vocabulary():
273         """The verdict vocabulary must NOT contain 'irrelevant'/'useless' (attribution
contract)."""
274         forbidden = {"irrelevant", "useless", "dead", "remove"}
275         for v in ablation.VERDICT_LABEL.values():
276             assert not any(bad in v.lower() for bad in forbidden)

```

```

277
278
279     def test_report_round_trips_json():
280         """The structured report is JSON-able and sorted by ?F1 desc (contribution
ranking)."""
281         videos = _learned_videos(6)
282         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
283                                                         signals=ablation.default_signals(),
284                                                         granularity="group"))
285         blob = json.dumps(report.as_dict()) # must not raise
286         back = json.loads(blob)
287         deltas = [r["delta_f1"] for r in back["rows"]]
288         assert deltas == sorted(deltas, reverse=True) # sorted desc
289         assert back["num_videos"] == 6
290         assert back["label_set_hash"] # provenance present
291
292
293     def test_seed_is_deterministic():
294         """Two runs with the same seed produce identical CIs (deterministic bootstrap)."""
295         videos = _learned_videos(6)
296         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
297                             granularity="group", seed=7)
298         r1 = run_ablation(videos, cfg)
299         r2 = run_ablation(videos, cfg)
300         assert [r.ci_low for r in r1.rows] == [r.ci_low for r in r2.rows]
301         assert [r.ci_high for r in r1.rows] == [r.ci_high for r in r2.rows]
302
303
304     # ----- table +
PDF
305
306
307     def test_format_table_is_sorted_and_readable():
308         videos = _learned_videos(6)
309         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
310                                                         signals=ablation.default_signals(),
311                                                         granularity="group"))
312         out = ablation.format_table(report)
313         assert "Ablation report" in out
314         assert "?F1(full?ablated)" in out
315         assert "verdict" in out
316         # the highest-?F1 signal name appears before the lowest in the rendered order
317         top = report.sorted_rows()[0].signal
318         bottom = report.sorted_rows()[-1].signal
319         assert out.index(top) < out.index(bottom)
320
321
322     def test_export_pdf_is_valid_nontrivial(tmp_path):
323         """A valid PDF with at least one page is produced, and it contains the verdict
legend."""
324         pytest.importorskip("reportlab")
325         pypdf = pytest.importorskip("pypdf")
326         videos = _learned_videos(6)
327         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
328                                                         signals=ablation.default_signals(),
329                                                         granularity="group"))
330         out = str(tmp_path / "ablation.pdf")
331         ablation.export_pdf(report, out)
332         assert os.path.isfile(out)
333         assert os.path.getsize(out) > 1000 # non-trivial
334         reader = pypdf.PdfReader(out)
335         assert len(reader.pages) > 0
336         text = "".join(p.extract_text() for p in reader.pages)
337         assert "Ablation" in text

```



```

338         assert "clears-bar" in text                                # the verdict key rendered
339
340
341     # ----- corpus loader
342     (no GPU)
343
344     def test_load_videos_tolerates_missing_audio(tmp_path):
345         """An older feature JSON without an ``audio`` block still loads ? audio columns
346         default to
347         0.0 (loader never crashes on a pre-audio JSON)."""
348         cand = {"start": 0.0, "end": 10.0,
349                 "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
350                 "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
351                 "label": 1}
352         data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
353                 "candidates": [cand], "gts": [[0.0, 10.0]]}
354         with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
355             json.dump(data, f)
356         videos = ablation.load_videos(str(tmp_path))
357         assert len(videos) == 1
358         assert videos[0].features["audio_hit_count"] == [0.0]          # defaulted, not
359         crashed
360     # ----- the
361     golden litmus
362
363     GOLDEN_DIR = os.path.join(
364         os.path.dirname(os.path.abspath(__file__)), "output")
365
366     def _golden_present() -> bool:
367         return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
368
369
370     @pytest.mark.skipif(not _golden_present(),
371                         reason="golden output/*_golden_features.json absent (run
372     fusion_golden first)")
373     def test_litmus_reproduces_gate_a():
374         """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
375         measured Gate-A
376         result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
377         tolerance.
378         This validates the whole framework against the recorded ground truth. Skips cleanly
379         in CI
380         where the golden substrate is absent; runs on the owner box where it is present."""
381         videos = ablation.load_videos(GOLDEN_DIR)
382         assert len(videos) == 6
383         cfg = ablation.litmus_config()
384         report = run_ablation(videos, cfg)
385         assert len(report.rows) == 1
386         row = report.rows[0]
387         assert row.signal == "gate_a"
388         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
389         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
390         assert row.sign_wins == 6 and row.sign_losses == 0
391         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
392         assert row.ci_low == pytest.approx(0.042, abs=0.01)
393         assert row.ci_high == pytest.approx(0.104, abs=0.01)
394         assert row.ci_excludes_zero is True
395         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
396         assert row.verdict == ablation.STRUCTURAL_PROTECTED
397         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
398         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)

```

```

395         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
396
397
398     @pytest.mark.skipif(not _golden_present(),
399                          reason="golden output/*_golden_features.json absent")
400     def test_litmus_report_is_well_formed():
401         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
402         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
403         assert isinstance(report, AblationReport)
404         assert report.num_videos == 6
405         assert report.full_set_signals == ["gate_a"]
406         assert report.label_set_hash
407         out = ablation.format_table(report)
408         assert "gate_a" in out
409
410
411     # ----- edge
cases
412
413
414     def test_min_players_gate_knob_zeroed():
415         """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
416         full = Variant(name="full", min_players=2, feature_names=["a"])
417         ablated = ablation._ablated_variant(full, (), "min_players")
418         assert ablated.min_players == 0
419         assert ablated.feature_names == ["a"]          # columns untouched
420
421
422     def test_ablated_label_falls_back_to_gate_knob():
423         """With no columns, the ablated variant name carries the gate-knob label, not
'?'."""
424         full = Variant(name="full", min_crossings=2)
425         ablated = ablation._ablated_variant(full, (), "min_crossings")
426         assert "min_crossings" in ablated.name
427
428
429     # -----
CLI
430
431
432     def _write_learned_corpus(d: str, n: int = 6) -> None:
433         for k in range(n):
434             cand = []
435             for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
((50.0, 51.0), 0)]:
436                 shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
437                 person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
438                 person["players_in_court"] = 4.0 if pos else 0.0
439                 person["in_court_ratio"] = 1.0 if pos else 0.0
440                 audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
441                 cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
442                             "audio": audio, "label": pos})
443                 data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
444                         "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
445                 with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
as f:
446                     json.dump(data, f)
447
448
449     def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
450         """``main()`` runs the LOGO ablation and writes both a PDF and the structured
JSON."""
451         pytest.importorskip("reportlab")
452         pytest.importorskip("pypdf")

```

```

453     d = str(tmp_path / "feat")
454     os.makedirs(d)
455     _write_learned_corpus(d)
456     pdf = str(tmp_path / "out.pdf")
457     js = str(tmp_path / "out.json")
458     monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
"group",
459                                     "--pdf", pdf, "--json", js])
460     ablation.main()
461     out = capsys.readouterr().out
462     assert "Ablation report" in out
463     assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
464     assert os.path.isfile(js)
465     report = json.loads(open(js, encoding="utf-8").read())
466     assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
"audio"}
467
468
469     def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):
470         """`--litmus` runs the static Gate-A config (single structural row)."""
471         d = str(tmp_path / "feat")
472         os.makedirs(d)
473         _write_learned_corpus(d)
474         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
475         ablation.main()
476         out = capsys.readouterr().out
477         assert "gate_a" in out
478         assert "static" in out                                     # litmus full set is the static
gate variant
479
480
481     def test_cli_main_needs_two_videos(tmp_path, monkeypatch):
482         d = str(tmp_path / "feat")
483         os.makedirs(d)
484         _write_learned_corpus(d, n=1)
485         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d])
486         with pytest.raises(SystemExit):
487             ablation.main()
488

```

13. user (2026-06-16T03:06:12.235Z)

```

total 28
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 01:52 ..
-rw-r--r-- 1 avidu 197609 4696 Jun 14 16:28 experiment_leaderboard.json
-rw-r--r-- 1 avidu 197609 1977 Jun 10 17:43 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609  689 Jun 10 15:57 heuristic_lovo_n6.json
---GITIGNORE---

```

Byte-compiled / optimized / DLL files

```

__pycache__/
*.py[cod]
*$py.class
*.pt

```

Local environment secrets

```

.env
*.env
.env.*

```

C extensions

```

*.so

```

Distribution / packaging

```
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
share/python-wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST
```

SQLite databases & Local output files

```
*.db
*.db-journal
*.db-wal
output/
output_dir/
backend/static/highlights_*.mp4
backend/static/*_compiled.mp4
```

OS generated files

```
.DS_Store
Thumbs.db
desktop.ini
```

Coverage data

```
.coverage
.coverage.*
htmlcov/
.pytest_cache/
```

Local Claude Code config (machine-specific)

```
.claude/
.█c█o█v█e█r█a█g█e█
█
█
```

Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from ruff + pytest; never meant to be committed ? keep them out of `git status` too).

```
scratch/
.venv/
```

14. assistant (2026-06-16T03:06:17.901Z)

I now have the full grounding. Let me check the regression_baseline.json reference (it's loaded by regression.py but I didn't see it in the listing) and the experiment_leaderboard schema to understand whether GT intervals appear in committed files today.

15. user (2026-06-16T03:06:20.845Z)

```
=== experiment_leaderboard.json (head) ===
[
```

```

{
  "timestamp": "2026-06-14T06:23:51.507393+00:00",
  "iou": 0.5,
  "label_set_hash": "50d98ffbc370",
  "num_videos": 6,
  "variant_a": {
    "name": "shuttle_only",
    "spec_hash": "47c867c36f2f",
    "aggregate": {
      "precision": 0.31807603780927995,
      "recall": 0.3101947096042718,
      "f1": 0.3068611057861412,
      "mean_iou": 0.5791831889148886
    }
  },
  "variant_b": {
    "name": "shuttle_person_fusion",
    "spec_hash": "253f4709b4c4",
    "aggregate": {
      "precision": 0.3617228001510626,
      "recall": 0.2708939086387243,
      "f1": 0.2863398846047612,
      "mean_iou": 0.5690111885714347
    }
  },
  "delta": {
    "precision": 0.04364676234178266,
    "recall": -0.0393008009655475,
    "f1": -0.02052122118138,
    "mean_iou": -0.01017200034345389
  },
  "honest_delta_f1": {
    "mean_delta": -0.02052122118137999,
    "ci_low": -0.16462806145542527,
    "ci_high": 0.16224446866060574,
    "n": 6.0,
    "ci_excludes_zero": false,
    "sign_test": {
      "wins": 2.0,
      "losses": 4.0,
      "ties": 0.0,
      "n_effective": 6.0,
      "p_value": 0.6875
    }
  }
},
{
  "timestamp": "2026-06-14T06:24:56.487711+00:00",
  "iou": 0.5,
  "label_set_hash": "50d98ffbc370",
  "num_videos": 6,
  "variant_a": {
    "name": "shuttle_person",
    "spec_hash": "ee89f1e41604",
    "aggregate": {
      "precision": 0.3617228001510626,
      "recall": 0.2708939086387243,
      "f1": 0.2863398846047612,
      "mean_iou": 0.5690111885714347
    }
  },
  "variant_b": {
    "name": "shuttle_person_audio",
    "spec_hash": "d68c39eba87c",
    "aggregate": {

```

```

        "precision": 0.4530798177857001,
        "recall": 0.3408138120839734,
        "f1": 0.3655826552797436,
        "mean_iou": 0.7074802852140344
    }
},
"delta": {
    "precision":
=== heuristic_lovo_n6.json ===
{
    "name": "heuristic_velocity_windowing_LOVO",
    "description": "The honest cross-video floor the Gen-0 learned model must beat: calibrated
velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
    "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local --manifest
output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
    "mean_f1": 0.480,
    "std": 0.242,
    "per_video": {
        "adarsh_avi_singles": 0.751,
        "kushagra_singles": 0.157,
        "largetest_doubles": 0.737,
        "gbaaddy": 0.309,
        "testlarge_short": 0.267,
        "mahadevpura_singles": 0.657
    }
}
}

=== git ls-files eval_baselines ===
eval_baselines/experiment_leaderboard.json
eval_baselines/gemini_wrapper_2026-06-10.json
eval_baselines/heuristic_lovo_n6.json

```

16. assistant (2026-06-16T03:06:30.425Z)

This is a critical grounding find. The committed `heuristic\_lovo\_n6.json` **already leaks per-video human-readable names** (`adarsh\_avi\_singles`, `kushagra\_singles`, `mahadevpura\_singles`, `gbaaddy`) ? exactly the source-attribution the legal memo says must be stripped. So this isn't hypothetical; there's a live de-attribution issue in the committed precedent. I have enough to design.

Now let me think through the design with the precise codebase seams in hand. The key facts:

- `eval\_baselines/` is committed; it carries aggregate JSON + `label\_set\_hash` (a SHA, opaque) but `heuristic\_lovo\_n6.json` leaks player names in keys.
- `\*\_golden\_features.json` is the per-video feature substrate (shuttle + person + audio columns, `start`/`end` intervals, `gts`, plus a `name` field) ? kept private on OneDrive, gitignored `output/`.
- The skipif tests (`tests/test\_ablation.py::test\_litmus\_reproduces\_gate\_a`, `tests/test\_fusion\_compare.py`) glob `output/\*\_golden\_features.json` and skip cleanly when absent ? this is the existing "skip-not-fail when key/data absent" pattern.
- `video\_id` = MD5 of source file ? must never be a public key.

Let me verify the visualize read_me before producing a diagram, and confirm what git-crypt / sops / age availability looks like is irrelevant ? I'll design tool-agnostic. Let me produce the structured output now.

17. user (2026-06-16T03:06:30.986Z)

```
<persisted-output>
Output too large (61.5KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\tool-
results\toolu_017qkr4xrFAmmSsucENCbaP8.json
```

Preview (first 2KB):

```
[
```

```

{
  "type": "text",
  "text": "# Imagine ? Visual Creation Suite\n\n## Modules\nCall read_me again with the
modules parameter to load detailed guidance:\n- `diagram` ? SVG flowcharts, structural diagrams,
illustrative diagrams\n- `mockup` ? UI mockups, forms, cards, dashboards\n- `interactive` ?
interactive explainers with controls\n- `chart` ? charts, data analysis, geographic maps
(Char.js, D3 choropleth)\n- `art` ? illustration and generative art\nPick the closest fit. The
module includes all relevant design guidance.\n\n**Complexity budget ? hard limits:**\n- Box
subtitles: ?5 words. Detail goes in click-through (`sendPrompt`) or the prose below ? not the
box.\n- Colors: ?2 ramps per diagram. If colors encode meaning (states, tiers), add a 1-line
legend. Otherwise use one neutral ramp.\n- Horizontal tier: ?4 boxes at full width (~140px
each). 5+ boxes ? shrink to ?110px OR wrap to 2 rows OR split into overview + detail
diagrams.\n\nIf you catch yourself writing \"click to learn more\" in prose, the diagram itself
must ACTUALLY be sparse. Don't promise brevity then front-load everything.\n\n**Accessibility:**
For HTML widgets, begin with a visually-hidden `<h2 class=\"sr-only\">` containing a one-
sentence summary of the visualization for screen-reader users. (SVG widgets use `role=\"img\"`
with `<title>` and `<desc>` instead ? see SVG setup.)\n\nYou create rich visual content ? SVG
diagrams/illustrations and HTML interactive widgets ? that renders inline in conversation. The
best output feels like a natural extension of the chat.\n\n## Core Design System\n\nThese rules
apply to ALL use cases.\n\n### Philosophy\n- **Seamless**: Users shouldn't notice where
claude.ai ends and your widget begins.\n- **Flat**: No gradients, mesh backgrounds, noise
textures, or decorative effects. Clean flat surfaces.\n- **Compact**: Show the essential inline.
Explain the rest in text.\n- **Text goes in your response, visuals go in the tool** ? All
explanatory text, descripti
...
</persisted-output>

```

18. assistant (2026-06-16T03:06:35.526Z)

This is a design memo, not a visual deliverable; the StructuredOutput tool is what the orchestrator reads. I won't produce a widget (it wouldn't be captured by StructuredOutput and the task is a written design). Let me read the rest of the diagram guidance briefly to decide ? actually, the output schema has no field for a widget reference, so a diagram adds no value to the captured result. I'll skip it and deliver the design via StructuredOutput.

I have everything I need. Let me write the structured design.

19. user (2026-06-16T03:08:26.135Z)

Structured output provided successfully

20. assistant (2026-06-16T03:08:40.427Z)

I've delivered the design via StructuredOutput. Key findings worth flagging from the grounding pass:

****Live de-attribution bug found:**** `C:\Users\avidu\Projects\badminton-highlight-indexer\eval\_baselines\heuristic\_lovo\_n6.json` is ****already committed**** and its `per\_video` keys leak human-readable source/player names (`adarsh\_avi\_singles`, `kushagra\_singles`, `mahadevpura\_singles`, `gbaaddy`, `testlarge\_short`, `largetest\_doubles`). This directly violates the de-attribution rule in the legal memo and is a concrete, shipped leak ? not hypothetical. The design folds its remediation (replace with opaque surrogate ids, keep the mapping private) into the same change.

****Design summary delivered:****

- ****Tier-0**** (public, unencrypted, no key): de-attributed/synthetic fixtures committed under a new tracked `eval\_baselines/fixtures/`, schema-compatible with the existing `fusion\_compare.load\_videos` / `ablation.load\_videos` globbers so any cloner runs real characterization regression with no key.
- ****Tier-1**** (key-gated): default = OUT-OF-REPO + token via the existing OneDrive store and the `#175` `RALLY\_GOLDEN\_DIR` seam (keeps sensitive bytes out of git history, sidesteps the legal "redistribution" act); alternative = encrypted-in-repo via ****SOPS+age**** (chosen over git-crypt for per-recipient keys + diffable JSON + clean rotation; ****git-lfs+encryption rejected**** as wrong tool).

- ****Key-absent = clean skip****, reusing the proven ``skipif(_golden_present())`` pattern already in ``tests/test_ablation.py`` / ``tests/test_fusion_compare.py``; a separate unconditional Tier-0 job keeps every clone green.
- Explicit honesty: ****protobuf/serialization is not secrecy****, and ****Tier-1 crypto cannot hide content from a key-holder who runs the tests**** ? it is only a barrier against the no-key public, consistent with the lead's DRM/analog-hole constraint.
- ``video_id`` (MD5 of source) is structurally excluded from any committed fixture, with a pre-commit/CI guard against attribution leaks.

Relevant files (absolute):

- ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_compare.py``,
- ``...\backend\eval\ablation.py``, ``...\backend\eval\fusion_golden.py``,
- ``...\backend\eval\regression.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_ablation.py``,
- ``...\tests\test_fusion_compare.py``
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\eval_baselines\`` (tracked;
``heuristic_lovo_n6.json`` is the leak)
- ``C:\Users\avidu\Projects\badminton-highlight-indexer\docs\GOLDEN_DATA_SHARING.md``